ROTP-PUF-FLASHER: Browser-Based Secure Provisioning and PUF-Bound Encrypted Firmware Execution for ESP32 IoT Devices

Agustín Ahumada Ramírez[1] and Raziel César Campos Sánchez[1]

Instituto Nacional de Astrofísica, Óptica y Electrónica (INAOE), Tonantzintla, Puebla, México
contact@agustinahumada.com

Abstract. Provisioning IoT devices without exposing setup material remains an open practical problem: network credentials, server addresses, and device identities are routinely exposed during setup, and firmware modules are typically stored in cleartext on flash. We present ROTP-PUF-FLASHER, a tool that integrates (1) a browser-based provisioning flow using the Web Serial API to flash the MicroPython base firmware, upload the runtime modules, enroll a runtime SRAM-PUF helper, and store configuration as a PUF-bound encrypted envelope, and (2) PUF-bound encrypted firmware execution in MicroPython, where, at runtime, a loader in cleartext reconstructs the device's SRAM Physical Unclonable Function (PUF) response, derives a symmetric key via HKDF-SHA256, verifies an HMAC-SHA256 tag over the encrypted module, and decrypts it into RAM for execution, refusing to run on HMAC failure. We map the tool's security properties and remaining gaps to the OWASP IoT Top 10, NIST IR 8259A, and STRIDE, and validate execution, provisioning, and authentication on real ESP32 hardware: identity reconstruction completes in 2.23 s, secure execution in 2.47 s, tamper rejection in 2.46 s, and the encrypted firmware envelope adds 69 bytes (11.75%) of overhead over the plaintext module. We further characterize the PUF across three boards, showing near-ideal inter-device distance and cross-device rejection. The open limit is stated explicitly: the configuration and firmware keys depend on the PUF; server-mediated origin is future work.

Keywords: SRAM-PUF · IoT security · firmware encryption · MicroPython · ESP32 · browser provisioning · OWASP IoT Top 10

1 Introduction

Commercial ESP32-class devices are widely deployed in IoT applications but are commonly shipped without activated hardware security controls (eFuses, Secure Boot, or Flash Encryption). Two security weaknesses affect most real-world deployments in this setting.

Insecure provisioning. When a device is set up for the first time, the operator must supply network credentials, server address, and device-specific configuration. Standard practice embeds these in the firmware image or transfers them

over an uncontrolled USB or serial connection, leaving them readable to anyone who later inspects the device. To our knowledge, no open-source tool treats this setup as a security-relevant flow backed by a hardware-rooted device identity.

Unprotected firmware modules. Application logic, communication protocols, and credentials stored in MicroPython scripts remain in cleartext on flash, readable and reusable without authorization. Prior work by our group demonstrated the feasibility of HMAC-verified encrypted execution on ESP32/MicroPython [18]; the present paper extends this into a provisioning tool with browser-based setup and multi-framework security evaluation.

Contribution. We present **ROTP-PUF-FLASHER**, a tool and evaluation framework with three concrete contributions:

1. A *browser-based provisioning flow* (the Web Flasher; Web Serial API, Chrome or Edge) that flashes the MicroPython base firmware (`ESP32_GENERIC` v1.25.0) with an erase-first policy, uploads the runtime `.py` modules through the MicroPython raw REPL (read-eval-print loop), enrolls the runtime PUF helper when needed, and writes `config.json` as a PUF-bound encrypted envelope. *Note:* encrypted payload build remains a host-side CLI step.
2. A *PUF-bound encrypted firmware execution* mechanism in MicroPython: a small loader in cleartext reconstructs the RTC SLOW SRAM-PUF response (the SRAM in the ESP32 low-power real-time-clock (RTC) domain), derives a symmetric key via HKDF-SHA256, verifies an HMAC-SHA256 tag over the ciphertext, and decrypts the AES-256-CBC payload into RAM for execution. In the demonstrated `.py` execution path, the protected module is decrypted directly into RAM and is never written to persistent storage in plaintext form.
3. A *multi-framework security evaluation* mapping the tool's security properties against the OWASP IoT Top 10 [16], NIST IR 8259A [14], and STRIDE [21], with measured performance results on real hardware.

Open problem (stated explicitly). The decryption key is derived exclusively from the PUF. An adversary who reconstructs the PUF, which is possible on a device without Secure Boot if the boot path is physically replaced, obtains the same key. Moreover, during provisioning and MicroPython execution the browser or host obtains derived key material to build encrypted artifacts. The architecturally correct solution requires server participation in key and configuration origin; Sect. 7 details this limit, which is reserved for thesis/journal work, not claimed here.

The rest of the paper reviews background and related work (Sect. 2), describes the threat model and design (Sect. 3), details the implementation (Sect. 4), presents the security and performance evaluations (Sects. 5 and 6), and closes with limitations and conclusions (Sects. 7 and 8).

2 Background and Related Work

2.1 SRAM Physical Unclonable Functions

SRAM cells power up to a manufacturing-determined preferred state, a unique and reproducible fingerprint [7] that must be stable enough to reproduce on the same chip yet differ across chips to prevent transplantation [6]. Because SRAM responses are noisy, reliable key derivation requires error correction via a fuzzy extractor or helper-data mechanism [3]. Peer-reviewed reliability studies show that stability depends on cell selection policy, operating conditions, aging, and spatial correlation [1,19,24]. Remanence-based side-channel attacks on SRAM PUFs have been demonstrated [25]; we treat remanence decay as a real attack surface and do not claim resistance to cold-boot, timed power-down, or semi-invasive capture.

On ESP32, Staníček's `esp32_puflib` [22] provides enrollment, helper-data generation, and reconstruction targeting the RTC FAST SRAM and deep-sleep DATA SRAM. Our MicroPython adaptation targets *RTC SLOW* SRAM, the surface reachable by runtime register cycling under MicroPython (RTC FAST and deep-sleep DATA SRAM remain future work); this constrains it to laboratory control status and does not claim native `esp32_puflib` compatibility.

2.2 Encrypted Firmware Distribution

The IETF SUIT manifest model [13] authenticates firmware metadata, versioning, and payload digests for constrained devices; that update-delivery layer is complementary to this paper rather than a claimed contribution. Our focus is narrower: protecting selected MicroPython firmware modules with a PUF-derived key reconstructed on demand. Espressif's `esp_encrypted_img` [4], the closest official reference, supports ECIES-P256 but still depends on a device secret in eFuse or firmware, which this reversible (no-eFuse) design avoids. The broader IoT attestation literature supports hardware-rooted checking [12] but targets attestation services rather than PUF-bound encrypted execution.

2.3 Related Work

Table 1 compares representative work on secure firmware update, PUF-bound update or integrity, OS-level PUF support, blockchain-assisted update, and browser-device provisioning. These works address pieces of the problem, but none of the surveyed systems combines browser-based provisioning with PUF-bound encrypted firmware execution on ESP32/MicroPython or evaluates the implementation against OWASP IoT Top 10, NIST IR 8259A, and STRIDE.

3 System Architecture

3.1 Threat Model and Scope

The system targets ESP32-class devices operating without irreversible hardware security controls (no eFuses burned, no Secure Boot, no Flash Encryption).

Table 1. Comparison with related works. FW prot./PUF: firmware update, integrity, code dissemination, or execution protection, with PUF binding indicated where applicable. Browser prov.: browser-based interface used for provisioning or flashing. OWASP/NIST/STRIDE: simultaneous evaluation against all three frameworks.

Work (Year)	Hardware Platform	FW prot. / PUF binding	Browser provisioning	OWASP/ NIST/STRIDE
ROTP-PUF-FLASHER (2026)	ESP32-WROOM	✓ AES-256+HMAC[a]	✓ Web Serial[b]	✓ all three
Falas et al. [5] (2022)	FPGA	∼ secure update	–	–
Prada-Delgado et al. [17] (2017)	BLE SoC	✓ PUF update[c]	–	–
Su et al. [23] (2019)	CRFID	✓ PUF code[d]	–	–
Joshi et al. [8] (2023)	Unspecified[e]	∼ PUF+blockchain	–	–
Chen et al. [2] (2023)	IoT protocol	✓ PUF+FW integrity[f]	–	–
Kietzmann et al. [9] (2024)	ESP32, HiFive	– (OS-level PUF)	–	NIST SP 800-90 only
Schwinne & Pelzl [20] (2026)	ESP8266/ESP32	–	✓ browser[g]	–

[a] PUF-bound encrypted firmware execution in MicroPython. [b] Browser flash, runtime-module upload, encrypted configuration, negative tests, backend puzzle authentication, and accepted telemetry exercised on hardware. [c] PUF keys for secure firmware update. [d] SRAM-PUF code dissemination for CRFID devices. [e] Platform not specified in accessible text. [f] PUF authentication, firmware integrity, and update protocol. [g] Browser-to-device local communication; no PUF binding or firmware encryption.

This *reversible* design constraint is intentional: it preserves testability and reprogrammability and avoids permanent device commitment.

Attacker capabilities (in scope). The attacker can: (i) read SPI flash and copy public code, PUF helper data, firmware ciphertext, and the encrypted `config.json` envelope; (ii) replay an encrypted firmware envelope to another device; (iii) tamper with a ciphertext byte in an encrypted module; (iv) replay an older encrypted envelope on the same device. Under passive flash extraction, configuration credentials remain ciphertext; they are exposed only if the attacker also obtains runtime key material or replaces the loader (out of scope below).

Out of scope. The system does not defend against an attacker who physically replaces the MicroPython loader or the second-stage bootloader (nothing prevents this replacement in the absence of Secure Boot), performs invasive chip extraction, applies voltage or temperature fault injection, or mounts side-channel analysis of PUF reconstruction. These vectors require separate evaluation campaigns and are stated as limits, not hidden.

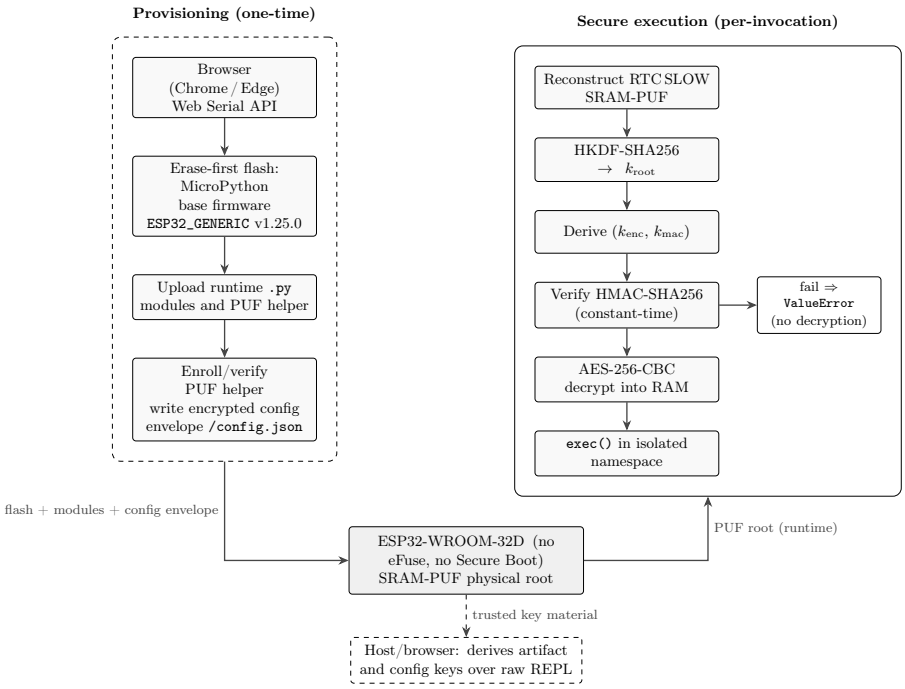

Fig. 1. ROTP-PUF-FLASHER system overview. Left (dashed): one-time provisioning via browser Web Serial, including PUF-helper enrollment and encrypted configuration injection. Right (solid): per-invocation PUF-bound encrypted MicroPython execution. The trusted host/browser derives artifact and configuration keys over raw REPL; the device independently reconstructs them from the enrolled helper at runtime.

3.2 Design Principles

Three principles govern the design.

Reversibility. No eFuse is burned; no irreversible hardware change is made. The device can be re-enrolled, re-flashed, or re-purposed.

Standard primitives only. AES-256-CBC [15] and SHA-256 are used via MicroPython's built-in `ucryptolib` and `uhashlib`; HMAC-SHA256 [10] and HKDF-SHA256 [11] are local RFC-compliant constructions over SHA-256. No proprietary or custom cryptographic primitives are introduced.

Fail-closed. The loader verifies the HMAC tag before any decryption. If verification fails, the encrypted module is not executed and no plaintext is produced.

3.3 Component Overview

Figure 1 shows the end-to-end flow. The provisioning phase (dashed box, left) runs once at device setup, entirely from the browser over Web Serial (detailed

in Sect. 4). The secure execution phase (solid box, right) runs on every payload invocation: the RTC SLOW PUF is reconstructed on the MicroPython device, a 32-byte symmetric key is derived, and the AES-256-CBC+HMAC-SHA256 envelope is verified and decrypted into RAM.

4 Implementation

4.1 PUF Enrollment and Key Derivation

Enrollment runs once per device, driven from the host over the raw REPL by the published enrollment scripts. The device samples the RTC SLOW SRAM region at address `0x50000000` ($N = 1000$, $s = 4096$ bytes per sample) using register cycling to expose unstable transient values, with a 50 ms power-off interval between samples that minimizes reconstruction noise (Sect. 6). It then selects 256 stable bits with a distributed-window policy (64 windows, 128 candidates per window) and protects them with a repetition-8 error-correcting code. These 256 selected, error-corrected bits feed HKDF-SHA256 to back the AES-256 key; their effective entropy is bounded by the selection policy and is not formally quantified here. The 32-bit selection from earlier memory-constrained trials is not a security configuration. Helper data (selected positions, ECC data, salt, context, mode) are stored in the ESP32 non-volatile storage (NVS) under namespace `rtc_puf` with a filesystem fallback at `/puf_helper.bin`.

Identity reconstruction for key derivation (`rtc_slow_puf_native.derive_key`) takes $A = 5$ voting attempts over fresh PUF samples, applies vote-majority reconstruction, and feeds the consensus material into HKDF-SHA256 [11]:

$$k_{\text{root}} = \text{HKDF-SHA256}\big(\mathit{material},\ \texttt{"DID-PUF-RTC-SLOW-v1"},\ \texttt{"puf\_identity\_key"} \,\|\, \mathit{nonce},\ 32\big), \tag{1}$$

with arguments in the order (input keying material, salt, info, output length), here and in Eq. 2; $\|$ denotes concatenation, and a null-byte separator (`\x00`) is inserted by `_info_bytes()` between the context and nonce strings. No key material is stored persistently; it is reconstructed from the PUF on each invocation.

4.2 Encrypted Firmware Envelope

The host tool `build_encrypted_firmware_from_device.py` fetches k_{root} from the device over the raw REPL, keeps it in host RAM, and derives two independent subkeys:

$$(k_{\text{enc}}, k_{\text{mac}}) = \text{HKDF-SHA256}\big(k_{\text{root}},\ \texttt{"DID-PUF-FIRMWARE-ENVELOPE-v1"},\ \texttt{"aes-256-cbc+hmac-sha256"},\ 64\big). \tag{2}$$

The envelope format `PUFENC1` is:

Field	Size	Description
Magic	8 B	`PUFENC1\0`
Version	1 B	Currently 1
Reserved	3 B	Zero
IV	16 B	Random initialization vector
Length	4 B	Ciphertext length (big-endian)
Ciphertext	ℓ B	AES-256-CBC(plaintext, k_{enc}, IV)
Tag	32 B	HMAC-SHA256(k_{mac}, header $\|$ ciphertext)

Total header overhead is 32 bytes; total envelope overhead is 32 + 32 + padding = 64+ bytes (65–80 bytes depending on the plaintext length modulo the AES block size). Measured on a 587-byte `protected_app.py` build: a 656-byte envelope (+69 bytes, 11.75%).

4.3 Secure Loader and Execution

The loader (`secure_firmware_loader.py`) remains in cleartext on the device. Its execution path implements a strict *verify-before-decrypt* policy:

1. Reconstruct the PUF and derive k_{root} via `derive_key()`.
2. Derive $(k_{\text{enc}}, k_{\text{mac}})$ from k_{root} (Eq. 2).
3. Read the encrypted envelope from `protected_app.enc`.
4. Parse and validate the `PUFENC1` header.
5. Compute the expected HMAC-SHA256 tag over `header` $\|$ `ciphertext`.
6. Compare tags in constant time (`_constant_time_equal`).
7. **If the tag does not match, raise `ValueError`; execution halts.**
8. Decrypt with AES-256-CBC and remove PKCS#7 padding.
9. Execute the plaintext module in an isolated namespace via `exec()`.

For the `.mpy` bytecode path, the decrypted bytes are written to a temporary root-level file, imported once, and then deleted from both the filesystem and `sys.modules` in a `finally` block.

4.4 Browser-Based Provisioning

The Web Flasher uses the Web Serial API (Chrome and Edge) to provision the MicroPython runtime end-to-end from the browser. It flashes the MicroPython base firmware (`ESP32_GENERIC-20250415-v1.25.0.bin`) with an erase-first policy, uploads the runtime `.py` modules as text through the raw REPL, enrolls the runtime PUF helper when absent, derives a configuration key with nonce `config-json-v1`, and stores `config.json` as an authenticated AES-256-CBC+HMAC-SHA256 envelope:

Listing 1. Plaintext configuration fields encrypted before storage.

```
{
  "device_id":  <integer id>,
```

```
  "api_key":    "<API credential>",
  "device_key_hex": "<64 hex chars>",
  "server_key_hex": "<64 hex chars>",
  "server_url": "http://<server-ip>",
  "server_port": 8000,
  "wifi_ssid":  "<2.4GHz SSID>",
  "wifi_pass":  "<password>",
  "read_interval_s": 30,
  "location":   "<deployment label>"
}
```

The tool also scans Wi-Fi networks, offers a filtering serial monitor with exportable logs, manages device files, and generates a provisioning report for laboratory tracking. A hardware smoke test on an ESP32-WROOM-32D exercised the browser base-image flash, runtime-module upload, encrypted `config.json` deployment, reboot-load, tamper rejection, plaintext-config rejection, and final restore. A heap-collection fix then allowed Wi-Fi activation, NTP synchronization, backend puzzle authentication (HTTP 200), token acquisition, and four telemetry cycles accepted by the server (HTTP 201), confirmed by MongoDB readings and Redis session state.

Scope note. The Web Flasher handles MicroPython runtime provisioning only (base firmware flash, module upload, PUF-helper enrollment, and encrypted configuration injection). Encrypted payload build and secure execution remain separate from the browser tool. Raw REPL uploads are text-based; binary PUFENC1 envelopes remain a CLI-operated step. The encrypted `config.json` protects configuration at rest against passive flash reads, but not against physical loader replacement without Secure Boot (Sect. 7).

4.5 Update Delivery Scope

The prototype protects modules after they are provisioned, but it does not claim a complete software-update mechanism. Authenticated remote delivery, versioned manifests, transport security, and server-side admission remain future work.

5 Security Evaluation

We evaluate ROTP-PUF-FLASHER against three complementary security frameworks. OWASP IoT Top 10 is a practitioner checklist rather than a formally peer-reviewed standard; we use it as a structured framing tool alongside the normative NIST IR 8259A baseline and the STRIDE methodology. Each row carries an evidence label: **P** (prototype, lab control), **Prop** (proposed, not yet implemented), **B** (blocked by design constraint); – marks out-of-scope categories; combined labels apply per sub-mechanism. Rows marked * share the open problem that the decryption key depends on the PUF alone (Sect. 7).

5.1 OWASP IoT Top 10

Table 2 maps each OWASP IoT Top 10 category to the corresponding security mechanism in ROTP-PUF-FLASHER.

Table 2. OWASP IoT Top 10 (2018) mapping.

#	Category	How addressed	Status
I1	Weak/Hardcoded Passwords	Credentials injected at provisioning, not hardcoded in firmware; `config.json` stored as a PUF-bound envelope (I7)	**P**
I2	Insecure Network Services	Outside this scope; provisioning reduces endpoint exposure	–
I3	Insecure Ecosystem Interfaces	Web Serial local interface; no exposed cloud API	**P**
I4	Lack of Secure Update	Not addressed by the current paper; authenticated update delivery remains future work	–
I5	Insecure/Outdated Components	Standard current primitives; pinned MicroPython v1.25.0 base image; no formal vulnerability-management process	**P**
I6	Insufficient Privacy Protection	Not addressed in this scope	–
I7	Insecure Data Transfer/Storage	Protected module ciphertext and PUF-bound `config.json` envelope on flash; plaintext exists in RAM/browser during use	**P***
I8	Lack of Device Management	Demo backend authentication observed; lifecycle/admission remains future work	**Prop**
I9	Insecure Default Settings	Provisioning requires explicit configuration; loader fail-closed on HMAC failure	**P**
I10	Lack of Physical Hardening	Reversible design excludes eFuses; physical loader replacement not prevented	**B**

5.2 NIST IR 8259A Baseline

Table 3 evaluates the tool against the six core device cybersecurity capabilities defined in NIST IR 8259A.

5.3 STRIDE Threat Analysis

Table 4 analyzes each STRIDE threat category against the provisioning and secure execution flow.

Table 3. NIST IR 8259A core device cybersecurity capability baseline.

Capability	How addressed	Status
Device Identification	PUF-reconstructed identity from silicon; three-board uniqueness evidence; backend puzzle authentication observed	**P**
Device Configuration	`config.json` encrypted from the browser and written atomically; Wi-Fi scan-assisted selection	**P***
Data Protection	Protected module ciphertext on flash; PUF-bound configuration envelope; plaintext confined to runtime paths	**P***
Logical Access to Interfaces	Web Serial local; no exposed network service during provisioning	**P**
Software Update	Not implemented in this paper; authenticated remote update delivery is future work	**Prop**
Cybersecurity State Awareness	Lifecycle/admission states proposed; audit logs are future work	**Prop**

5.4 Adversarial Test Results

The demo check script (`run_encrypted_mpy_demo_check.py`) runs a positive control and a tamper case on the enrolled device (a restored-positive control confirms recovery); timings were taken at the host (Sect. 6). Table 5 summarizes the two timed cases.

The tamper case confirmed fail-closed behavior on the real board: the loader raised `ValueError` before any decryption began. Tamper rejection (2.46 s) costs essentially the same as successful execution (2.47 s): both reconstruct the PUF first, and that reconstruction dominates the runtime, so the security property here is *verify-before-decrypt* correctness, not a timing advantage. Cross-device and wrong-PUF-helper replay were also exercised and rejected in every tested direction across three boards (Sect. 6). The remaining adversarial gap is a downgrade test, pending the monotonic version counter discussed in Sect. 7.

6 Performance Evaluation

All timing measurements were taken on board A, an ESP32-WROOM-32D (ESP32-D0WD-V3, 240 MHz, 520 KB SRAM) running the MicroPython v1.25.0 base image of Sect. 4, connected via USB-serial; the PUF quality characterization extends to three boards. Timing was measured end-to-end at the host using Python's `time.monotonic()` across the full serial round-trip including file upload and REPL execution.

6.1 Timing Measurements

Table 6 summarizes the measured costs.

Table 4. STRIDE analysis of the provisioning and secure execution flow.

Threat	Mechanism and evidence	Status
Spoofing	PUF-bound configuration plus backend puzzle authentication; server-mediated key origin still future work	**P***
Tampering	HMAC-SHA256 verified in constant time before any decryption; tamper rejection measured at 2.46 s	**P**
Repudiation	Not addressed; append-only lifecycle log is future work	–
Information Disclosure	Passive flash reads expose module/config ciphertext; loader replacement can still expose runtime material	**P***
Denial of Service	Not addressed in this scope	–
Elevation of Privilege	Boot-path integrity is blocked without Secure Boot; server-mediated key origin remains future work	**B**/**Prop**

Table 5. Adversarial test results on ESP32-WROOM-32D; both cases ran sequentially on the same enrolled device.

Case	Result	Time (s)
Positive (own PUF key)	`PROTECTED_APP_MPY_OK`	2.47
Tamper (1-byte XOR flip)	`HMAC verification failed`	2.46

The end-to-end build (7.53 s) is dominated by device round-trips for key derivation (7.52 s), not by the cryptographic computation (0.01 s); the host tool performs several device-side reconstructions in one raw-REPL session, and consolidating them would bring the build close to the per-invocation cost (about 2.2 s).

6.2 PUF Quality Under Lab Conditions

We characterized the PUF on three ESP32-D0WD-V3 boards in the complete configuration (Table 7): boards A and B are WROOM-32D modules; board C is a D1 R32 module (same silicon, different form factor and USB-serial bridge). Reconstruction is stable on all three boards: the worst same-boot post-ECC error is a single corrected bit (0.0098%) on each, and reconstruction was accepted on every same-boot, reset, and physical power-cycle trial, always yielding the same per-device key (ten same-boot and eight hardware-reset reconstructions per board, plus two physical USB power cycles with four reconstructions each on boards A and B). Inter-device uniqueness is near the 50% ideal (raw inter-chip Hamming distance 49.19% A–B, 49.74% A–C, 49.86% B–C), and cross-device replay was rejected in all five tested directions (another board's helper gives ≈36% error, `accepted=False`, no key), so a stolen helper does not reconstruct

Table 6. Performance on ESP32-WROOM-32D, MicroPython v1.25.0. Per-operation times are medians of five runs (variation under 1%, apart from a higher cold-start reconstruction); enrollment is from its one-time setup log and is excluded from per-invocation cost. Wall-clock times include USB-serial latency and are independently rounded.

Operation	Time (s)	Notes
PUF enrollment (setup only)	445.9	1000 samples, 256 bits, size = 4096 B; not per-boot
PUF identity reconstruction	2.23	$A = 5$ vote-majority attempts; zero bit errors
Envelope build (pure crypto)	0.01	AES-256-CBC + HMAC-SHA256 over 587 B; host-only
Envelope build (total)	7.53	Includes device round-trips for key derivation
Secure firmware execution	2.47	PUF recon + HMAC verify + AES decrypt + exec in RAM (`.py` path)
Tamper rejection	2.46	HMAC fails before decryption; fail-closed
Envelope overhead	+69 B (11.75%)	587 B plaintext → 656 B envelope

Table 7. PUF reconstruction quality on three ESP32-D0WD-V3 boards (A, B: WROOM-32D; C: D1 R32), complete configuration. Percentages are over the 10 240 bits sampled per reconstruction (2048 helper positions, five voting attempts). Worst-case post-ECC error stays far below the native bar (0.15%) and the 5% policy; reconstruction was accepted on every trial.

Metric	Board A	Board B	Board C
Same-boot worst error (post-ECC)	0.0098%	0.0098%	0.0098%
Post-reset worst error (post-ECC)	0.0098%	0.0684%	0.0391%
Uniformity (Hamming weight)	50.2%	50.0%	49.5%

the key elsewhere. These are nominal room-temperature results for the MicroPython RTC SLOW PUF (the laboratory control of Sect. 7): temperature, voltage, aging, and a larger population remain future work. Enrollment used a 50 ms power-off interval; the loader reconstructs at the default 10 ms and tolerates the mismatch (worst error 0.0195%, zero residual errors). The complete configuration enrolls with ≈128 KB of free heap; a tighter setup (≈108 KB) instead raised a `MemoryError`, a setup-dependent limit, not a property of the algorithm.

7 Discussion and Limitations

Central Open Problem: Key Depends on the PUF Alone. The decryption key k_{root} is derived exclusively from the PUF response (Eq. 1). An adversary who reconstructs the PUF (feasible without Secure Boot) obtains the same key and can

decrypt the protected modules. Furthermore, the host obtains k_{root} over the raw REPL to build the payload, and the browser obtains derived configuration-key material during provisioning, so both are trusted parties in the current architecture. The architecturally correct solution is server-mediated key and configuration origin, with the server participating in key generation and releasing operational configuration only after online authentication; this central thesis extension is reserved for future publication.

MicroPython Is a Laboratory Control. The RTC SLOW adaptation does not claim equivalence with the native `esp32_puflib`; the Python loader is a research prototype, not a production security mechanism. The symmetric construction is standard and the 256-bit selection backs a nominally full-strength key (a formal entropy analysis is pending), but production use still needs per-build nonces with rollback protection (a fixed nonce leaves old envelopes replayable until a monotonic version counter is added), key-buffer zeroization, an envelope size cap, server-side payload signing independent of the PUF key, and host handling that never exposes the root key; the demonstrated `.py` path (RAM-only) is preferred over the `.mpy` path, which briefly writes plaintext to disk.

Web Flasher Hardware Smoke and Backend E2E. The hardware smoke and backend exercise of Sect. 4 surfaced two practical issues: a heap shortage that initially blocked Wi-Fi driver activation (fixed with explicit garbage collection before Wi-Fi initialization) and an encrypted-configuration server key that had to be realigned with the selected FastAPI backend. After both fixes, the device completed puzzle authentication, received a token, sent temperature and humidity telemetry, and the server accepted the readings with HTTP 201 while preserving an active Redis session.

No Secure Boot or Flash Encryption. Without Secure Boot, an attacker with physical write access can replace the loader itself, bypassing all encryption checks and extracting PUF-derived material at runtime. Without Flash Encryption, public code, helper data, and ciphertext envelopes remain readable from flash; the protected module and `config.json` remain ciphertext under passive extraction, but not under active boot-path replacement. These are intentional limits of the reversible, no-eFuse line, not oversights.

8 Conclusion and Future Work

We presented ROTP-PUF-FLASHER, a tool integrating browser-based controlled provisioning, PUF-bound encrypted configuration storage, and PUF-bound encrypted MicroPython firmware execution for ESP32-class IoT devices, evaluated against OWASP IoT Top 10, NIST IR 8259A, and STRIDE. On a real ESP32-WROOM-32D board the per-invocation operations complete in about 2.5 s with a +69-byte (11.75%) firmware-envelope overhead, the fail-closed loader rejects a single-byte tamper before decryption, the Web Flasher path

reaches backend puzzle authentication and accepted telemetry (repeated three consecutive times on two WROOM-32D boards), and the PUF was characterized across three boards (inter-device Hamming distance near 50%, cross-device rejection in every tested direction). Prior work covers the surrounding threads separately; among the surveyed works, none combines browser-based provisioning with PUF-bound encrypted execution or applies these three frameworks simultaneously to a PUF implementation on commodity ESP32 hardware.

The central open problem, that key material still depends on the PUF and is exposed to a trusted host or browser during artifact construction, is stated explicitly and deferred to the thesis. Future work includes server-mediated key and configuration origin via authenticated key exchange, a lifecycle/admission server with revocation, authenticated remote OTA with TLS and signed manifests, post-quantum manifest signatures, and environmental PUF characterization over a larger board population.

Reproducibility. Source code, measurement scripts, pre-compiled binaries, and documentation: https://github.com/agustinra24/rotp-puf-flasher-congreso.

Acknowledgments. The authors thank INAOE's Master's program in Security Science and Technology for the laboratory infrastructure.

Disclosure of Interests. R.C. Campos Sánchez coauthored the prior work cited as [18]; the authors have no other competing interests relevant to this article.

References

1. Baturone, I., Prada-Delgado, M.A., Eiroa, S.: Improved generation of identifiers, secret keys, and random numbers from SRAMs. IEEE Trans. Inf. Forensics Security **10**(12), 2653–2668 (2015)
2. Chen, Z., et al.: FSMFA: Efficient firmware-secure multi-factor authentication protocol for IoT devices. Internet of Things **21**, 100685 (2023)
3. Dodis, Y., Ostrovsky, R., Reyzin, L., Smith, A.: Fuzzy extractors: How to generate strong keys from biometrics and other noisy data. SIAM J. Comput. **38**(1), 97–139 (2008)
4. Espressif Systems: esp_encrypted_img: Encrypted image component for ESP-IDF. https://components.espressif.com/components/espressif/esp_encrypted_img (2026)
5. Falas, S., Konstantinou, C., Michael, M.K.: A modular end-to-end framework for secure firmware updates on embedded systems. ACM J. Emerg. Technol. Comput. Syst. **18**(1), 3:1–3:19 (2022)
6. Gao, Y., Al-Sarawi, S.F., Abbott, D.: Physical unclonable functions. Nat. Electron. **3**(2), 81–91 (2020)
7. Holcomb, D.E., Burleson, W.P., Fu, K.: Power-up SRAM state as an identifying fingerprint and source of true random numbers. IEEE Trans. Comput. **58**(9), 1198–1210 (2009)
8. Joshi, H., Anand, A., Kokila, J.: Secure firmware update architecture for IoT devices using blockchain and PUF. In: Proc. IEEE iQ-CCHESS (2023)

9. Kietzmann, P., Schmidt, T.C., Wählisch, M.: PUF for the commons: Enhancing embedded security on the OS level. IEEE Trans. Dependable Secure Comput. **21**(4), 2194–2210 (2024)
10. Krawczyk, H., Bellare, M., Canetti, R.: HMAC: Keyed-hashing for message authentication. RFC 2104, IETF (1997)
11. Krawczyk, H., Eronen, P.: HMAC-based extract-and-expand key derivation function (HKDF). RFC 5869, IETF (2010)
12. Kuang, B., Fu, A., Susilo, W., Yu, S., Gao, Y.: A survey of remote attestation in internet of things: Attacks, countermeasures, and prospects. Comput. Secur. **112**, 102498 (2022)
13. Moran, B., Tschofenig, H., Birkholz, H.: A manifest information model for firmware updates in internet of things (IoT) devices. RFC 9124, IETF (2022)
14. National Institute of Standards and Technology: IoT device cybersecurity capability core baseline. NISTIR 8259A, NIST (2020)
15. National Institute of Standards and Technology: Advanced encryption standard (AES). FIPS 197, NIST (2023)
16. OWASP Foundation: OWASP IoT top 10. https://owasp.org/www-project-internet-of-things/ (2018)
17. Prada-Delgado, M.A., Vázquez-Reyes, A., Baturone, I.: Trustworthy firmware update for Internet-of-Thing devices using physical unclonable functions. In: Proc. IEEE Global Internet of Things Summit (GIoTS). pp. 1–5 (2017)
18. Salinas Victorino, A., Campos Sánchez, R.C., Zapata Lara, J.d.J.: Encrypted-firmware runtime execution with HMAC verification on ESP32/MicroPython. Tech. rep., INAOE (2026)
19. Sarazá-Canflanca, P., Carrasco-López, H., Santana-Andreo, A., Brox, P., Castro-López, R., Roca, E., Fernández, F.V.: Improving the reliability of SRAM-based PUFs under varying operation conditions and aging degradation. Microelectron. Reliab. **118**, 114049 (2021)
20. Schwinne, C., Pelzl, J.: Secure local communication between browser clients and resource-constrained embedded IoT devices. J. Cybersecur. Priv. **6**(1), 9 (2026)
21. Shostack, A.: Threat Modeling: Designing for Security. Wiley (2014)
22. Staníček, O.: esp32_puflib: Open SRAM-PUF library for ESP32. GitHub repository, https://github.com/staniond/esp32_puflib (2022)
23. Su, Y., et al.: SecuCode: Intrinsic PUF entangled secure wireless code dissemination for computational RFID devices. IEEE Trans. Dependable Secure Comput. **18**(4), 1699–1717 (2019)
24. Wilde, F., Gammel, B.M., Pehl, M.: Spatial correlation analysis on physical unclonable functions. IEEE Trans. Inf. Forensics Security **13**(6), 1468–1480 (2018)
25. Zeitouni, S., Oren, Y., Wachsmann, C., Koeberl, P., Sadeghi, A.R.: Remanence decay side-channel: The PUF case. IEEE Trans. Inf. Forensics Security **11**(6), 1106–1116 (2016)